2026학년도 2학기 중간고사 문제유형 정답지

[1주차]

문항	정답
1	**D** (변수의 타입이 컴파일 시점에 결정되며, 컴파일러가 타입 일치 여부를 검사한다.)
2	**D** ('Value'의 약자로, 한 번 초기화되면 재할당이 불가능한 읽기 전용 변수를 선언한다.)
3	**D** (Kotlin에서 변수 선언(예: val a = 5)은 값을 반환하는 '식'이다. — *변수 선언은 '문(Statement)'임*)
4	**C** (String)
5	**A** (`val arr = IntArray(5)`)
6	**리터럴 (Literal)**
7	**O**
8	`val` 로 선언된 변수는 읽기 전용(불변, Immutable)이므로 한 번 초기화된 이후 값을 재할당(변경)할 수 없기 때문이다.
9	`: Long` (완성 코드: `val bigNumber: Long = 100` 또는 `100L`)
10	`it` (완성 코드: `val numbers = IntArray(5) { it }`)

[2주차]

문항	정답
1	**O** (단, Enum 클래스나 Sealed 클래스의 모든 경우를 처리하는 경우는 예외적으로 생략 가능)
2	`class` 는 여러 개의 인스턴스(객체)를 생성할 수 있는 설계도인 반면, `object` 는 클래스 정의와 동시에 단 하나의 인스턴스만 생성하는 싱글톤(Singleton) 객체를 만든다는 점이다.
3	**A** (Parameter는 함수 정의 시 선언하는 변수 이름, Argument는 함수 호출 시 전달하는 실제 값이다.)
4	**D** (동일한 타입의 인수를 개수 제한 없이(가변적으로) 받기 위해)
5	**A** (해당 클래스 내부에서만 접근할 수 있다.)
6	**A** (주생성자 (Primary Constructor))
7	**A** (Unit)
8	**B** ((1) `$` (2) `${ }`)
9	**D** (블록 바디 (Block Body))
10	**B** (쉼표(`,`))
11	`level: Int` (완성 코드: `fun startGame(user: String, level: Int = 1)`)

문항	정답
12	`printInfo(age = 30, name = "홍길동")`
13	`1..10` (완성 코드: `for (i in 1..10)`)
14	**B** ((1) `val` (2) `radius` (3) `Double`)
15	`p1.sayHello()`

16	**Circle 클래스 UML 다이어그램** (접근제한자: `+` public, `-` private)
	`+-------------------------------+`
	`|  Circle  |`
	`+-------------------------------+`
	`| + radius: Double  |`
	`| - pi: Double  |`
	`+-------------------------------+`
	`| + circumference(): Double  |`
	`| + area(): Double  |`
	`| + diameter(): Double  |`
	`+-------------------------------+`

[3주차]

문항	정답
1	**D** (ordinal)
2	**D** (`!!` (Not-Null Assertion))
3	**D** (부생성자 (Secondary Constructor))
4	**D** (`when (x) { ... }` 표현식 — *실행문은 init 블록이나 함수 내부에 위치해야 함*)
5	**C** (주생성자 프로퍼티 초기화 -> init 블록 -> 부생성자 본문)
6	**init 블록 (초기화 블록, Initializer Block)**
7	**컴파일 시점 (Compile-time)**
8	해당 Enum 클래스에 정의된 **모든 열거형 상수(분기관)를 빠짐없이 처리**했을 경우
9	`valueOf()` (예: `Color.valueOf("RED")`)
10	**Smart Cast (스마트 캐스트)**
11	`?.` (완성 코드: `val userName = user?.name`)
12	`?:` (완성 코드: `val userName: String = name ?: "Guest"`)
13	`enum` (완성 코드: `enum class Color`)
14	`this` (완성 코드: `constructor(age: Int) : this("Unknown", age)`)
15	`this` (완성 코드: `println("User created: ${this.name}")`)

[4주차]

문항	정답
1	**C** (`(Int, Int) -> String`)
2	**B** (고차 함수 (Higher-Order Function))
3	**A** (List는 읽기 전용(불변)이며, MutableList는 요소 추가/삭제/변경이 가능하다.)
4	**D** (기존 리스트와 새 요소가 포함된 새로운 리스트가 생성되어 반환된다.)
5	함수(람다식)가 정의될 당시의 외부 스코프(환경)에 있는 변수(자유 변수)를 포획(Capture)하여, 외부 함수가 종료된 이후에도 해당 변수에 접근하고 수정할 수 있게 하는 기능(또는 그 함수와 환경의 조합)
6	`y` (람다식의 매개변수 `x`가 아닌 외부 스코프에서 참조되는 변수)
7	`items.map { it + 2 }`
8	`it`
9	`val y1 = fun(x: Int): Double { return x * 2.0 }` (또는 표현식 바디: `val y1 = fun(x: Int): Double = x * 2.0`)
10	**함수 참조 (Function Reference)** (또는 멤버 참조 / 호출 가능 참조, Callable Reference)
11	`[ ]` (인덱스 접근 연산자, 예: `list[0]`)
12	`it % 2 == 0` (완성 코드: `val evenNumbers = numbers.filter { it % 2 == 0 }`)
13	`(Int) -> Int`
14	`-` (완성 코드: `val listB = listA - "B"`)
15	`(String) -> Unit` (완성 코드: `fun runAction(action: (String) -> Unit)`)

[5주차]

문항	정답
1	**A** (List는 중복 요소를 허용하지만 Set은 중복 요소를 허용하지 않는다.)
2	**D** (`get()` 은 키가 없으면 null을 반환하고, `getValue()` 는 NoSuchElementException 예외를 발생시킨다.)
3	`[3, 5]` (`filter { it > 2 }` -> `[5, 3, 7]` -> `sorted()` -> `[3, 5, 7]` -> `take(2)` -> `[3, 5]`)
4	**D** (`associateWith()`)
5	**B** (`{a=[apple, ant], b=[banana]}`)
6	`"USA" to "Washington"`
7	`map()`

문항	정답
8	`setOf()`
9	`getOrDefault()`
10	`associate()`
11	`contains()` (또는 `in` 연산자)
12	`it.value` (완성 코드: `val highScores = scores.filter { it.value > 20 }`)
13	`put` (완성 코드: `map.put(3, "C")`)
14	`associateBy` (완성 코드: `val map = people.associateBy { it.name }`)
15	`0` (완성 코드: `.withDefault { 0 }`)

[6주차]

문항	정답
1	**C** (특정 클래스의 멤버 함수인 것처럼 호출할 수 있는 함수를 클래스 외부에서 정의하는 기능이다.)
2	**수신 객체 (Receiver Object)** (확장 함수가 호출된 대상 객체 / 범위 함수가 적용된 컨텍스트 객체)
3	**C** (반드시 getter 함수를 정의해야 한다.)
4	**수신 객체 (Receiver Object)**
5	함수 본문 내부에서 `this == null` (또는 안전 호출 연산자 `this?.`, 엘비스 연산자 `this ?: ...`)을 사용하여 확인한다.
6	`let`
7	`apply`
8	`fun Int.isOdd(): Boolean`
9	`let` (완성 코드: `name?.let { println("Name: $it") }`)
10	`lastChar` (완성 코드: `val String.lastChar: Char`)
11	(1) `Int`, (2) `isEven` (완성 코드: `fun Int.isEven(): Boolean`)
12	`this` (완성 코드: `if (this == null) return 0`)
13	`?: 0` (※ 문맥상 `name?.run { this.length } ?: 0` 형태에서 null일 경우 0을 반환하도록 엘비스 연산자 `?: 0` 사용)
14	`apply`는 **수신 객체(컨텍스트 객체) 자신**을 반환하는 반면, `run`은 **람다식의 마지막 실행 결과(표현식 값)**를 반환한다.
15	두 함수 모두 블록 내부에서 대상 객체(수신 객체)에 접근할 때 **`this` 키워드**를 사용(또는 생략하여 멤버에 직접 접근)한다는 점이다.

[7주차]

문항	정답
1	**B** (클래스는 인스턴스화할 수 있지만, 인터페이스는 그 자체로 직접 인스턴스화할 수 없다.)
2	**B** (타입 검사(is) 이후, 사용 시점 이전에 다른 스레드에 의해 값이 (다른 타입으로) 변경될 수 있기 때문에)
3	**C** (가장 안쪽 수신자(b)가 바깥쪽 수신자(a)를 가린다(Shadowing).)
4	**B** (ClassCastException 예외(Exception)가 발생한다.)
5	**C** (값을 직접 할당할 순 없지만, get()을 이용한 기본 구현은 제공할 수 있다.)
6	**함수 오버로딩 (Function Overloading / Method Overloading)**
7	**하위 타입 (Subtype)** (또는 자식 클래스 / 서브클래스, Subclass)
8	**B** (섀도잉 (Shadowing))
9	`null`
10	`is` (완성 코드: `if (obj is String)`)
11	`as?` (완성 코드: `val str: String? = obj as? String`)
12	`Clickable, Movable` (완성 코드: `class Person : Clickable, Movable`)
13	`{ }` (빈 중괄호 블록 — 완성 코드: `fun onClick() { }`)
14	`get() = "Guest"`
15	**O** (리스코프 치환 원칙 / 다형성)